# Cost-Aware Multi-Model Inference Router (CAMIR)

Project Proposal

Abhishek Darji  Aniket Anil Naik  Tamizh Selvan Manivannan

San José State University — MS Software Engineering

Advisor: Prof. Vijay Eranti

October 27, 2026

**Abstract**

Routing and cascading send easy prompts to a small language model and hard ones to a large model. CAMIR builds and evaluates such a router on a single Apple M4 laptop, with local small and medium tiers and a hosted large tier. The project makes four contributions: (1) a measured account of the *cost of escalation itself*—the cost, delay and energy wasted when a small-model attempt is rejected—and policies that reduce it; (2) a new family of escalation *confidence signals*, including signals read from a mixture-of-experts router; (3) *calibrated routing with guarantees*, a risk-controlled threshold that bounds the quality lost relative to always using the large model, for policies that skip or abort the small attempt; and (4) *per-step routing in agentic workflows*, where switching tiers mid-trajectory carries its own cost. For each contribution we state what prior work already establishes and what remains open. Two extensions sharpen the first contribution: *verification-based escalation*, where the large model scores the small model's answer in one prefill-only pass instead of regenerating it, and a *regime map* that tells a practitioner when cascading, pre-routing or always using the large model wins. Supporting studies test the policies under concurrent load and on open-ended chat traffic, make the guarantee adaptive to distribution shift, and release the trace dataset.

## 1 Introduction

Large hosted models are accurate but costly and slow; small local models are cheap and fast but fail more often. Pre-generation routers [2, 4] and confidence-gated cascades [1, 5, 6, 8] try to capture the savings, and cascade routing [3] combines the two. Vendors now ship routers as products [24, 25, 26, 27], and API gateways such as LiteLLM include a pre-generation tier router [23]. The project runs on a MacBook M4 with a pay-per-token API for the large tier: no HPC or cloud credits are needed.

## 2 Problem Statement

Given a stream of prompts (and, in the agentic setting, multi-step tasks) and a pool of tiers (small, medium, large), choose for each prompt or step which tier answers, so that quality stays within $\epsilon$ of always using the large tier while cost, latency (mean and tail) and energy are minimized, and so that the quality statement is backed by a statistical guarantee rather than a tuned threshold. Published evaluations mostly report token cost or the share of calls sent to the large model [1, 2, 7]; surveys note the lack of shared metrics, pools and cost bases [17] and of non-financial costs [18].

# 3 Contributions and Novelty

**Honest positioning.** Several of these topics are already active. Bouchard [7] characterizes threshold cascades decision-theoretically and finds that a pre-generation router often beats the best cascade, attributed mainly to avoiding the cheap model's cost. Conformal methods for routing and cascading already exist [9, 10, 11]. Budget-aware per-step routing for agents exists [13] (and [14], which we have not yet read). We therefore state, for each contribution, a narrower gap that we have not found covered in the sources we read; our literature search is not exhaustive and each claim is to be re-checked with the advisor before submission.

## 3.1 C1: The cost of escalation itself

**Established.** A cascade pays for every model it queries [1, 10], and the cheap model's cost explains much of the router's advantage in token cost [7].

**Our contribution.** With $c_S, c_L$ the tier costs, $g$ the gate cost, $e$ the escalation rate and $\rho = (c_S + g)/c_L$, a cascade saves

$$\text{Savings} = (1 - e)\Delta - eW,$$
$$\Delta = \mathbb{E}[c_L - c_S - g \mid \text{acc}], \quad W = \mathbb{E}[c_S + g \mid \text{esc}],$$

breaking even at $e^* = \Delta/(\Delta + W)$ $(= 1 - \rho$ for homogeneous costs). Per prompt, attempt the small model first only if the predicted acceptance probability $\hat{a}(x) > \rho(x)$. For latency, escalated requests pay $t_S + t_g + t_L$, so under idealized assumptions a cascade's p95 beats always-large only if $e < 5\%$. These relations are elementary; the contribution is (i) measuring the tax in dollars, seconds (mean and tail) and joules on real hardware with a local-plus-hosted pool, and (ii) two policies built from it: **P3**, a skip-rule route-then-cascade, and **P4**, an early-abort cascade that abandons the small attempt when the running mean log-probability falls below a threshold after $n_0$ tokens.

**Extension X1: verification-based escalation (policy P5).** In a standard cascade, escalation makes the large model regenerate the answer from scratch, so every rejected attempt is pure waste. P5 changes what escalation does. A verifier tier $V$ reads the prompt and the small model's answer in a single forward pass and scores the answer by the mean log-probability it assigns to it, $v(x, y_S)$. If $v \geq \tau_v$ the small answer is kept; otherwise the large model regenerates. The verifier is the hosted large model if its API returns log-probabilities for supplied text, and otherwise the local medium tier. Verification is prefill-only (no token-by-token decode), so its cost $c_V$ is typically below $c_L$ for long answers. Let $v_{\text{pass}}$ be the fraction of escalated requests that pass verification. An escalated request then costs $c_S + g + c_V + (1 - v_{\text{pass}})c_L$ instead of $c_S + g + c_L$, so P5 helps exactly when $v_{\text{pass}}\, c_L > c_V$; it attacks the cases where the small answer was right but the gate was unsure. The idea is related to speculative decoding [21], where a large model verifies a small model's draft in parallel; the differences are that we verify whole answers rather than draft tokens, that acceptance is approximate (its quality loss is controlled by C3, not by an exactness argument), and that the verifier may sit across an API boundary. In the sources we read we have not found answer-level verification charged inside a cascade cost model. Feasibility depends on the log-probability access in the hosted API, which is checked in spike S6; the verifier can also be a local tier, which makes the medium tier a requirement. A verifier that agrees too easily with wrong answers is a real risk, and E10 measures it.

**Extension X2: a regime map.** The break-even relations give a decision rule that practitioners need before they build anything. For each point in the plane of cost ratio $\rho$ and escalation rate $e$, and for a stated latency target, we compute which strategy is cheapest at matched quality

(always-large, pre-generation router, cascade, P3, P4, P5), using the measured $\Delta$, $W$ and latency distributions, and we validate the predicted regions by replaying the traces. The deliverable is a figure and a small calculator that takes a team's own $e$, $\rho$ and latency target. A map that says "do not cascade here" is as useful as one that says "do".

### 3.2 C2: A new confidence signal

**Established.** Token log-probability, top-1/top-2 margin (with isotonic calibration [8]), sampled-answer agreement [19, 6], self-verification [5], and probes on hidden states or prompt embeddings are used to decide escalation.

**Gap.** Mixture-of-experts (MoE) routers produce a per-token distribution over experts. Prior work studies router uncertainty to improve MoE fine-tuning [15], and, in the sources we read, does not use it as a signal for whether an answer is correct or should be escalated.

**Our contribution.** We test the hypothesis that expert-routing statistics (normalized router entropy, its change across layers, and expert-selection instability) predict small-tier correctness. The small tier would be a small open MoE model (candidates such as OLMoE [16], to be confirmed in a feasibility spike for MLX support and router access). The signal is available at prefill and at decode time at near-zero extra cost, which makes the skip rule and early abort of C1 cheaper to apply. It is compared on AUROC and calibration error against log-prob, margin, self-consistency, and a learned hidden-state probe (our fallback if no suitable MoE model runs locally). A negative result is a reportable outcome.

### 3.3 C3: Calibrated routing with guarantees

**Established.** Conformal Cascade [9] defers when a conformal prediction set is not a singleton, with a coverage guarantee (shown on multiple-choice benchmarks; open-ended generation left for future work). RouteNLP [10] gives a marginal conformal guarantee on a violation probability. RACER [11] gives distribution-free risk control for set-valued routing. Conformal risk control [12] is the general tool.

**Gap.** We have not found a guarantee that covers the full *tax-aware* policy (skip, early abort and gate together), and generation tasks (math word problems, code with unit tests) are less covered than multiple-choice ones.

**Our contribution.** Let $A(\lambda)$ be the event that the small answer is accepted under threshold $\lambda$ after any skip or abort rule, and define the loss $\ell(\lambda) = \mathbf{1}[A(\lambda) \wedge S \text{ wrong} \wedge L \text{ right}]$, which is non-increasing in $\lambda$ and zero when the small answer is not accepted. Since the error of the cascade minus the error of always-large is at most $\ell$, choosing $\hat{\lambda}$ by conformal risk control on a calibration split gives $\mathbb{E}[\ell(\hat{\lambda})] \leq \alpha$, i.e. an expected quality loss relative to always-large of at most $\alpha$. The guarantee is marginal (not per prompt), assumes exchangeability, and weakens under distribution shift; we test coverage empirically with repeated random splits and under a deliberate shift (calibrating on one dataset and testing on another). With P5, acceptance $A(\lambda)$ also includes passing verification; we use a one-parameter family in which the verification threshold is tied to $\lambda$, so that acceptance stays monotone in $\lambda$ and the same argument applies.

**Extension X3: adaptive recalibration.** A fixed threshold calibrated once is fragile when the prompt mix drifts. We add an online update in the spirit of adaptive conformal inference [22]: a random fraction of requests is audited by also running the large tier, which reveals the loss $\ell$, and the working risk level is adjusted after each audited request. The audit rate is a cost, so we report it, and we measure how quickly coverage recovers after a deliberate shift (calibrate on science questions, then stream code and math). We do not claim the finite-sample marginal guarantee for

this variant; what we test is whether the long-run average of the audited loss stays near $\alpha$ under shift, and whether an analogous formal statement holds is left to be checked with the advisor.

### 3.4 C4: Per-step routing in agentic workflows

**Established.** Budget-aware per-step routing for multi-step agents, trained with boundary-guided reinforcement learning, reduces dollar cost on agent benchmarks [13]; its cost is token dollars and router overhead is excluded from the reported metrics.

**Gap.** In an agent loop the context grows each step, so moving a step from the local tier to the hosted tier re-sends the whole history (the hosted model must prefill it again), and each tier switch adds latency. We have not found this switching cost modeled or measured on a local-plus-hosted pool.

**Our contribution.** We extend the C1 model to a trajectory: the cost of escalating at step $t$ includes the wasted small-model step and a prefill term that grows with context length, and a policy chooses the tier at each step. We evaluate on a lightweight multi-step tool-use benchmark that fits a laptop (candidates: function-calling data such as xLAM, or a small multi-step subset of an agent benchmark; feasibility is checked in a spike). We compare per-step policies (always-small, always-large, per-step threshold from C3, a learned step router in the style of [13]) on task success, dollars, seconds and joules.

### 3.5 Supporting studies and release

**Behavior under load.** A single-stream laptop study misses queueing delay, which often dominates tail latency in serving. We run a trace-driven discrete-event simulation of concurrent requests, using the measured service-time distributions of each tier, a concurrency or rate cap on the hosted tier, and a single local stream on the M4, and report how the escalation tax shifts p95 and p99 latency at several arrival rates. A small live run with several concurrent clients on a subset checks the simulation.

**Open-ended traffic.** The four benchmarks have checkable answers, which real traffic often lacks, and published router evaluations show that routing behaves differently on chat prompts [23]. We add a slice of about 200 open-ended chat prompts from a public chat dataset (a candidate is WildChat; its license is checked first). Quality is judged by blinded pairwise comparison against the large model's answer, with a judge from a different model family, both presentation orders, and conflicting verdicts recorded as ties; the judge's self-disagreement is reported. The guarantee on this slice uses the judged loss and is therefore weaker than on the verifiable sets.

**Trace release.** The trace dataset (outputs, log-probabilities, verification scores, timings, energy, and useful-versus-wasted labels) and the replay harness are released, subject to the licenses of the benchmarks and the hosted provider's terms on redistributing outputs, which are checked before release.

### 3.6 Priority and fallback

**Core:** C1 with X1 and X2, and C3. They build directly on the measurement pipeline. **Supporting:** the load study, the open-ended slice, X3, and the trace release. **Higher risk:** C2 and C4. If no MoE model runs locally, C2 falls back to a hidden-state probe; if no suitable agent benchmark runs on the laptop, C4 is restricted to multi-step chains assembled from the same datasets; if the hosted API cannot score supplied text, X1 uses the local medium tier as verifier, so the medium tier is kept. If time runs short, scope is cut in this order: C4, C2, X3, the open-ended slice, the load

study. C1, C3, X1 and X2 are not cut. Whether the extensions beat the baselines is open; null results will be reported.

| Contribution | Closest prior work | Open gap addressed |
|---|---|---|
| C1 Escalation cost (+X1, X2) | [1, 7, 10, 21, 23] | Latency, tail, energy on local+hosted pool; tax-aware policies; answer-level verification instead of regeneration; regime map |
| C2 Confidence signal | [8, 19, 5, 15] | MoE router statistics as escalation signal |
| C3 Guarantees (+X3) | [9, 10, 11, 12, 22] | Guarantee for skip/abort/verify policies; generation tasks; audited recalibration under shift |
| C4 Agentic per-step | [13, 14] | Switching/re-prefill cost across steps; measured latency and energy |

## 4 Architecture

Client layer (dashboard, offline harness) → FastAPI router service (ingestion and cache, feature extractor, policy engine, confidence-signal module, conformal calibrator, cost and latency accountant, and a step controller for agent loops) → tiers (small and medium MLX-LM 4-bit local, large hosted API) → trace store (SQLite/Parquet). Evaluation is trace-driven: outputs, signals, log-probs, timings and energy are cached per prompt (or per step) and tier, policies are replayed offline, and a live subset validates replay fidelity. Caching every tier's output for every prompt is also what makes the counterfactual losses in C3 computable. For P5 the traces also store the verifier's score of each small answer, and for the load study they store each tier's service-time distribution.

## 5 Two Paths

**Path A (local, medium tier)** Small and medium local tiers; routing between them; baselines; proof of concept; signal study (C2) on local models; started in CMPE 295A.

**Path B (large tier)** Adds the hosted large tier and real cost accounting; the guarantee experiments (C3), tax-aware policies and agentic extension (C4); completed in CMPE 295B.

Both paths share the same code, trace format and harness.

## 6 Features

- Pluggable policies and baselines (always-S/M/L, random, feature router, a complexity tier router modelled on LiteLLM and re-implemented from its documentation, cascades, cascade routing, oracle, P3, P4, P5, per-step policies).
- A regime-map calculator and a load simulator driven by the same traces.
- Confidence signals: mean log-prob, minimum token probability, margin, self-consistency, self-verification, MoE router statistics, hidden-state probe.
- Conformal calibrator that outputs a threshold and its stated guarantee for a chosen $\alpha$.

- Per-request and per-step accounting of cost, TTFT/TPOT/end-to-end time and energy, split into useful and wasted work.
- Trace store, replay harness, statistics (bootstrap intervals, paired tests), frontier plots, and a web dashboard.

## 7 Requirements

- Hardware: Apple M4 MacBook (variant and RAM recorded in setup). Hosted large-tier API with a small spending cap.
- Software: MLX-LM, llama.cpp/Ollama (fallback), FastAPI, SQLite, Parquet, optional Redis, a sandbox for HumanEval.
- Models: Qwen2.5 small ≈1.5B, medium 7B (4-bit), large 72B hosted [20]; a small open MoE model for C2; Llama 3.x fallback.
- Data: ARC-Easy, MMLU, GSM8K, HumanEval, about 1,400–1,500 stratified prompts split 40/20/40 into router-training, calibration and test sets; a multi-step tool-use set for C4; about 200 open-ended chat prompts and a judge model from a different family for the open-ended slice.

## 8 Evaluation Plan

E1 per-tier characterization; E2 signal quality (AUROC, calibration error) for all signals including C2; E3 tax measurement; E4 policy frontiers; E5 sensitivity to $\rho$; E6 tail latency; E7 replay-versus-live fidelity; E8 guarantee coverage (violation rate against $\alpha$ over repeated splits and under dataset shift); E9 agentic per-step policies (task success, cost, time, energy, switching overhead); E10 verification-based escalation (P5 against P2–P4: pass rate $v_{\text{pass}}$, $c_V/c_L$, wrong answers accepted by the verifier, net savings); E11 regime map (predicted strategy regions against replayed ones); E12 behavior under load (p95/p99 against arrival rate, simulation against a small live run); E13 open-ended chat slice (gate quality and policy frontiers without ground truth, judge agreement); E8 is extended with adaptive recalibration under shift and its audit cost. Quality must be within $\epsilon$ (2–5 points) of always-large. Dollars, joules and seconds are reported separately. Threats: thermal throttling, benchmark contamination, API variance, replay fidelity, small calibration sets. No results exist yet.

## 9 Timeline (two semesters)

| Weeks | Milestone |
|---|---|
| 295A 1–3 | Abstract, formation presentation, topic and advisor sign-off |
| 295A 3–9 | Literature and baselines; feasibility spikes (log-probs, hosted tier, energy, verifier access S6, MoE model access and agent benchmark S4); architecture and design |

| Weeks | Milestone |
|---|---|
| 295A 9–15 | Functional specifications; Path A proof of concept: full trace collection including verification scores and the chat slice, scorers, replay engine, baselines, E1 and the replay-versus-live check; report chapters 1–2 |
| 295B 1–7 | Live router; accountant; tax measurement (E3); P3 and P4; conformal calibrator |
| 295B 5–10 | P5 verification-based escalation (E10); coverage experiments and adaptive recalibration (E8); regime map (E11); open-ended slice (E13) |
| 295B 8–12 | Load study (E12); MoE signal study (C2) and agentic extension (C4) if time allows; full evaluation |
| 295B 11–14 | Testing, deployment, performance analysis; trace release; report; advisor reviews |
| 295B 14–15 | Presentation, poster, Project Exposition, final report |

## 10 Deliverables

Router service; trace dataset with useful-versus-wasted labels; evaluation harness; tax-aware policies including verification-based escalation; regime-map calculator; load simulator; confidence-signal library including the MoE-router signal; conformal calibrator with adaptive recalibration; agentic step router; dashboard; analysis report; paper draft; open-source release.

## 11 Risks

- Log-probs or MoE router outputs unavailable in MLX-LM: feasibility spikes; fall back to llama.cpp/Ollama, or to a hidden-state probe for C2.
- Hosted API price or availability changes: cache all large outputs; two providers.
- Thermal throttling and API variance distort latency: repetition, logging, cooldown.
- Overlap with fast-moving related work (conformal routing, agentic routing): re-check the literature monthly and narrow the claims if a gap closes.
- The hosted API cannot score supplied text, or the verifier accepts wrong answers too often: use the local medium tier as verifier and report the verifier's error rate (spike S6, E10).
- Judge noise on the open-ended slice: both presentation orders, ties for conflicts, a judge from another model family, and the judge's self-disagreement reported.
- Scope: four contributions plus three extensions and three supporting studies in two semesters is ambitious; the cut order is C4, C2, X3, the open-ended slice, the load study, and C1, C3, X1, X2 are protected.
- The tax proves small or routers equal cascades: report it as a finding.

## 12 Expected Outcomes

A measured account of when the cost of escalation makes cascading worse than routing in cost, latency and tail latency on commodity hardware, summarized as a regime map; a test of whether verifying the small answer instead of regenerating it lowers that cost; an evaluation of whether MoE router statistics are a useful escalation signal; thresholds with a stated, empirically tested quality guarantee for tax-aware policies; a first measurement of tier-switching cost in agent loops; and an open, reproducible system.

## References

[1] L. Chen, M. Zaharia, J. Zou. FrugalGPT. arXiv:2305.05176, 2023.

[2] I. Ong et al. RouteLLM. arXiv:2406.18665, 2024.

[3] J. Dekoninck, M. Baader, M. Vechev. A unified approach to routing and cascading for LLMs. ICML 2025. arXiv:2410.10347.

[4] D. Ding et al. Hybrid LLM. ICLR 2024. arXiv:2404.14618.

[5] P. Aggarwal et al. AutoMix. NeurIPS 2024. arXiv:2310.12963.

[6] M. Yue et al. LLM cascades with mixture of thought representations. ICLR 2024. arXiv:2310.03094.

[7] D. Bouchard. Is escalation worth it? A decision-theoretic characterization of LLM cascades. arXiv:2605.06350, 2026.

[8] V. Kotte. UCCI: Calibrated uncertainty for cost-optimal LLM cascade routing. arXiv:2605.18796, 2026.

[9] Y. Dou, S. Lian, S. Li. Conformal Cascade: Distribution-free accuracy guarantees for multi-tier LLM inference. arXiv:2607.25018, 2026.

[10] D. Guo, S. M. Yiu, J. Wu. RouteNLP: Closed-loop LLM routing with conformal cascading and distillation co-optimization. arXiv:2604.23577.

[11] S. Hao, H. Zeng, H. Wei, B. Jing. RACER: Risk-aware calibrated efficient routing for large language models. arXiv:2603.06616, 2026.

[12] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, T. Schuster. Conformal risk control. arXiv:2208.02814 (ICLR 2024).

[13] C. Zhang et al. Budget-aware agentic routing via boundary-guided training. arXiv:2602.21227, 2026.

[14] AgentRouter: Heterogeneous model routing for cost-optimal multi-step agentic workflows. arXiv:2609.22951 (not yet read; to be reviewed).

[15] H. Wu et al. GW-MoE: Resolving uncertainty in MoE router with global workspace theory. arXiv:2406.12375.

[16] N. Muennighoff et al. OLMoE: Open mixture-of-experts language models. arXiv:2409.02060, 2024.

[17] Y. Moslem, J. D. Kelleher. Dynamic model routing and cascading for efficient LLM inference: a survey. TMLR. arXiv:2603.04445.

[18] C. Varangot-Reille et al. Doing more with less: a survey on routing strategies for resource optimisation in LLM-based systems. arXiv preprint, 2025.

[19] X. Wang et al. Self-consistency improves chain of thought reasoning. ICLR 2023. arXiv:2203.11171.

[20] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

[21] Y. Leviathan, M. Kalman, Y. Matias. Fast inference from transformers via speculative decoding. ICML 2023. arXiv:2211.17192.

[22] I. Gibbs, E. Candès. Adaptive conformal inference under distribution shift. NeurIPS 2021. arXiv:2106.00170.

[23] BerriAI. LiteLLM documentation: Router and Auto Routing, and the auto-routing benchmark page. docs.litellm.ai (vendor-reported results; the auto router was documented as a beta feature when read).

[24] Amazon Web Services. Use Amazon Bedrock Intelligent Prompt Routing for cost and latency benefits. AWS Machine Learning Blog, April 22, 2025.

[25] Microsoft. Model router for Microsoft Foundry: How it works. Microsoft Learn documentation.

[26] NVIDIA. Deploying the NVIDIA AI Blueprint for cost-efficient LLM routing. NVIDIA Technical Blog, March 26, 2025.

[27] OpenRouter. Auto Router. OpenRouter documentation.